

Chapter 10. AI Governance

AI governance has become a central topic in recent years, as organizations try to balance fast progress in AI with the need for safety and responsibility.

On the one hand, companies have started taking a more cautious approach to customer-facing AI apps and begun investing in governance to avoid becoming the next negative headline. This shift was driven by several failures from early GenAI adoption without proper safeguards. [A well-known example](#) comes from New Zealand, where the supermarket chain Pak 'n' Save launched an app designed to help customers use up leftovers. Users could input ingredients they had at home, and the app would generate a recipe. The project backfired when it suggested combining household chemicals into chlorine gas and described it as “the perfect non-alcoholic beverage to quench your thirst and refresh your senses.” This episode highlighted the *reputational and safety risks* of unchecked AI.

Meanwhile, governments, particularly in Europe, are taking the lead in setting *AI regulations*. The introduction of the EU Artificial Intelligence Act (EU AI Act) has pushed companies to carefully evaluate which applications they develop and deploy and make sure they comply with stricter requirements.

Beyond regulation and reputation, there's also a hard business reality: today, AI is deeply embedded into critical applications across industries. A disruption in these systems can cause immediate and significant financial losses. This makes AI governance as much about *resilience and business continuity* as it is about compliance and ethics.

In this chapter, we will explore how Databricks can serve as a platform that supports the implementation of effective AI governance. In the previ-

ous chapters, we’ve already discussed managing data and models with Unity Catalog, applying guardrails, and monitoring system behavior. Now, we’ll connect those capabilities to regulatory frameworks.

We’ll look into the EU AI Act, which is the most comprehensive AI regulation at the moment, and we’ll examine what it requires and how following MLOps best practices on Databricks can help organizations meet those requirements in practice. To go deeper into understanding the EU AI Act and its implications, I recommend reading [*The AI Engineer’s Guide to Surviving the AI Act*](#) by Larysa Visengerlyeva (O’Reilly).

This chapter also introduces an *MLOps checklist*—an instrument organizations can use to assess their readiness. Before they promote systems to production, teams need to validate not only those systems’ technical performance and compliance but also their operational readiness. These checks ensure that deployment is not just technically feasible but also approved from a governance perspective, with clear accountability for who signs off on the release. You can also find the checklist in this book’s repository under the *ai_governance* folder.

The Scope of AI Governance

To understand the scope of AI governance, we need to begin with three foundations: the definition of what qualifies as an AI system, the classification of such systems by risk level, and the actors who bear responsibilities under the EU AI Act. Together, these elements set the boundaries for how governance applies in practice.

Definition of AI Under the EU AI Act

What is AI? The term is used broadly, from statistical models to GenAI applications. To avoid this ambiguity, the EU AI Act provides the following definition in [Chapter 1, Article 3](#):

“AI system” means a machine-based system that is designed to operate with varying levels of autonomy and that may exhibit adaptiveness after deployment, and that, for explicit or implicit objectives, infers, from the input it receives, how to generate outputs such as predictions, content, recommendations, or decisions that can influence physical or virtual environments.

The definition covers a wide range of AI techniques and approaches, including statistical methods, optimization theory, various machine learning models, and reinforcement learning.

The EU AI Act goes beyond defining AI systems in general. It also introduces a specific category for general-purpose AI (GPAI) models, which include widely known families such as OpenAI’s GPT, Anthropic’s Claude, and Meta’s Llama. It defines GPAI models as follows:

“General-purpose AI model” means an AI model, including where such an AI model is trained with a large amount of data using self-supervision at scale, that displays significant generality and is capable of competently performing a wide range of distinct tasks regardless of the way the model is placed on the market and that can be integrated into a variety of downstream systems or applications, except AI models that are used for research, development or prototyping activities before they are placed on the market.

The challenge of definition: A three-factor approach

The European Law Institute challenges the definition of an AI system in the EU AI Act, arguing that “Practically no single element of the definition in Article 3 No 1 of the AI Act is effective in clearly distinguishing AI systems from other systems.” At the same time, the Institute recognizes the necessity of drawing such distinctions and proposes a *three-factor approach* to more effectively define what constitutes an AI system.¹ This approach grades each of the proposed factors on a three-level scale (0, +, and ++), where 0 indicates that the factor is not present in a given system.

The details of these three factors are as follows:

Factor I: use of data or domain-specific expert knowledge in development

This factor acknowledges that a system is built using techniques that learn from data. It is absent (0) in systems such as general-purpose databases. It has medium representation (+) in analytics applications, where statistical analysis of historical data is applied and domain experts are consulted. It is strongly represented (++) in ML methods that rely on extensive training data to build and refine the system.

Factor II: creation of new know-how during operation

This factor is absent (0) in systems that only execute predefined calculations, such as deterministic algorithms and classification models that apply fixed rules rather than generating new know-how. This factor has medium representation (+) in optimization systems such as navigation tools, scheduling applications, and other computationally intensive but formally well-defined algorithms. It is strongly represented (++) in systems that generate new strategies or heuristics during use, guided by abstract objectives. Examples include online reinforcement learning systems, adaptive control systems for robotics, and online adaptive recommender systems.

Factor III: degree of formal indeterminacy of outputs

This factor is absent (0) in tasks with clearly defined outcomes, where each input yields a single correct result, such as mathematical calculations. It has medium representation (+) in systems with limited variability but objectively bounded results, such as handwriting recognition. It is strongly represented (++) in systems where outputs are inherently subjective and multiple results may all be seen as correct by different users. Examples include predicting loan default probabilities, classifying text, and generating content.

A system qualifies as an AI system if it scores at least three + signs across the three evaluation factors. [Table 10-1](#) illustrates examples of different systems and how they are assessed in ways that lead to decisions on

whether they can be classified as AI. Some cases are straightforward, while others depend on context. For instance, an optical character recognition (OCR) application can be considered an AI system when it's processing handwritten text, but not when it's dealing with typed text. Similarly, route planners and timetable generators may or may not qualify as AI, depending on whether they incorporate historical data and adaptive patterns. By contrast, use cases such as recommender systems and medical image recognition are clear examples of AI systems.

Table 10-1. Examples of AI applications and their classification as AI systems^a

Application	Factor I: use of data or domain-specific expert knowledge in development	Factor II: creation of new know-how during operation	Factor III: degree of formal indeterminacy of outputs	AI system?
OCR (typewritten text)	++	0	0	<i>No</i>
OCR (handwriting)	++	0	+	<i>Yes</i>
Search engine	++	0	++	<i>Yes</i>
Recommendation system	++	Varies from 0 to ++	++	<i>Yes</i>
LLM-based chatbot	++	Varies from 0 to ++	++	<i>Yes</i>
Route planner	0/+	+	+	<i>Yes/No</i>
Predictive maintenance	++	0	++	<i>Yes</i>
Medical image diagnostics	++	0	++	<i>Yes</i>
AI-assisted grammar correction	++	0	+	<i>Yes</i>
Timetable generator	0/+	+	+	<i>Yes/No</i>

^a Adapted from [the guidelines by the European Law Institute](#).

Not all AI systems are created equal. Some are tightly regulated because of the risks they pose, while others fall largely outside the scope of regulation.

Now that we've looked at examples of systems and whether they qualify as AI, we can turn to how the EU AI Act classifies them.

Risk Categories and Whom They Apply To

The EU AI Act takes a *risk-based approach*, meaning the higher the potential risk to human safety and fundamental rights, the stricter the rules. As of early 2026, the risk categories imposed by the EU AI Act have become the operational standard for any AI system entering the European market.

The high-level summary of the EU AI Act specifies four types of AI systems based on their risk. (Note that these are not formal categories.):²

Unacceptable risk

This category includes general social scoring systems that evaluate people's trustworthiness and exploit vulnerabilities. For example, it includes systems that lenders could use to systematically identify low-income individuals and offer predatory loans to them.

High risk

This category includes credit-scoring systems that determine loan or mortgage eligibility. It also includes systems used to detect insurance fraud and monitor suspicious transactions, plus hiring screening algorithms and AI that's used in medical devices.

Limited risk

This category includes chatbots, AIs that generate texts and images, and ecommerce systems that make recommendations based on user preferences.

Minimal risk

This includes most AI applications on the market, such as spam filters, AI-assisted grammar correction systems, and internal corporate applications like those that perform product demand forecasting.

As we alluded to previously, the EU AI Act does not formally define categories with these four names. Instead, the Act specifies prohibited AI practices, identifies two risk-based groups of systems, and says that all other AI practices and systems fall outside the Act's scope. These designated practices and systems are as follows:

Prohibited AI practices (in Article 5)

These are practices that create an unacceptable risk to fundamental rights and are strictly banned (e.g., social scoring, predatory behavioral manipulation).

High-risk AI systems (in Article 6)

These are systems that have a significant impact on health, safety, or fundamental rights. They are subject to the most intensive MLOps requirements, including mandatory risk management and data quality standards.

AI systems with specific transparency obligations (in Article 50)

The systems in this group are often referred to as “limited-risk” AI systems. They are not explicitly prohibited or high risk, but they interact directly with humans, so users must be informed that they are interacting with an AI system.

The EU AI Act also applies to the following:

Providers

These are the primary “builders” of AI, meaning the entities that develop AI systems or GPAI models and place them on the market under their own name or trademark. An example of a provider is Mistral, a French AI company that releases foundation models in the EU.

Deployers

These are the “users,” meaning organizations (public and private) that use AI systems under their authority for professional purposes. An example of a deployer would be a German city authority that deploys a third-party AI for traffic management and congestion prediction.

Non-EU providers and deployers

This group represents the global reach of the Act. If the output of an AI system (like a medical diagnosis or a credit score) is used within the EU, the Act applies regardless of where the company is headquartered. An example of a non-EU provider or deployer would be a Canadian company that supplies AI medical diagnosis tools that are used by French hospitals.

Importers and distributors

These are the companies that form the “AI supply chain,” meaning those that bring AI systems into the EU or make them available on the market. An example would be a Spanish distributor that sells AI-powered robots manufactured in Japan.

Product manufacturers

These are the “integrators,” meaning companies that embed AI into their physical products. An example of a product manufacturer would be a carmaker that manufactures vehicles with built-in AI driver-assistance systems.

Authorized representatives

These are the people and entities that act as the “legal bridge” under the Act. They reside in the EU and are appointed by non-EU providers to handle those providers’ legal obligations. An example of an authorized representative would be a consultancy in Brussels that serves as the EU representative for a US AI vendor.

Affected persons

These are the “citizens,” meaning individuals in the EU who are subject to AI’s influence. This group is the reason the Act exists; they are granted specific rights to explanations and protections. An example of an affected person would be a consumer in Italy who has been denied a loan based on an AI-driven credit assessment.

The majority of obligations stated in the regulation fall on deployers and providers of high-risk AI systems. According to Article 25, any distributor, importer, deployer, or other third party is considered to be a provider of a high-risk AI system if they do any of the following:

Rebrand the system

An example of this would be a company reselling an AI product under its own brand (a process called *white-labeling*) or using a vendor’s model but releasing it to customers under the company’s logo without clearly referencing the original provider.

Make a substantial modification to the system

An example of this would be making a significant change to a high-risk system that’s already on the market, such as changing a model’s behavior by retraining it or altering its key safety controls.

Change the intended purpose of the system

This would include repurposing an AI system that was originally considered to present minimal risk so that it now functions in a high-risk capacity. An example of this would be repurposing a general AI model for hiring decisions or credit scoring.

[According to Article 16](#), providers of high-risk AI systems must ensure that their high-risk systems are compliant with the requirements specified in Articles 9–15 of the EU AI Act. These articles aren’t just legal checkboxes; they represent the EU’s technical standards for risk management, data quality, and transparency. In the next section, we’ll examine those requirements and map them to the AI lifecycle stages, alongside the practices and tools introduced in earlier chapters.

It is also vital to remember that minimal-risk AI systems can pose material business risks. For example, a demand-forecasting system may be unregulated, but a small drop in its accuracy could cause multimillion-dollar losses. By applying these governance practices to all systems, organizations can build a level of operational resilience that will protect their bottom line, not just their legal team.

Requirements for High-Risk AI

Under the EU AI Act, providers of high-risk AI systems are not only required to meet technical standards but must also establish a *quality management system*, to ensure that compliance isn't just a one-time event but a continuous part of the AI's operational lifecycle.

The core technical requirements are laid out in Articles 9 through 15, which have become the blueprint for “trustworthy AI” in the enterprise:

- Article 9: Risk Management System
- Article 10: Data and Data Governance
- Article 11: Technical Documentation
- Article 12: Record-Keeping
- Article 13: Transparency and Provision of Information to Deployers
- Article 14: Human Oversight
- Article 15: Accuracy, Robustness, and Cybersecurity

The EU AI Act applies across the entire lifecycle of AI systems.

Before we examine Articles 9–15 in detail, we'll outline a typical AI project lifecycle. Then, within each article, we'll discuss the measures that should be applied at various stages of the lifecycle and highlight the tools that can support their implementation.

AI Project Lifecycle

In the previous chapters, we touched on parts of the development process, but implementing governance requires a more holistic view. Chip Huyen, in *[Designing Machine Learning Systems](#)* (O'Reilly), defines

several lifecycle stages that are essential for any ML system. While AI engineering follows a similar path, the transition from traditional ML to Generative AI introduces significant nuances. Here are the six stages of a project:

1. *Project scoping*: Developers define the project's scope, business objectives, and success criteria. They identify risks and constraints, secure stakeholder alignment, and allocate resources.
2. *Dataset/context engineering*: Developers prepare and process data for machine learning. This includes ingestion, cleaning, and data transformation (which we covered in Chapters [2](#) and [7](#)). Data annotation can also be considered as part of data engineering.
3. *Model development/adaptation*: Developers build and refine models through feature engineering, algorithm selection, and hyperparameter tuning (which we covered in [Chapter 3](#)).
4. *Deployment*: Developers operationalize models so their predictions are accessible to end users (which we covered in Chapters [4](#), [5](#), and [9](#)).
5. *Monitoring and continual learning*: Developers track deployed models for performance drift, detect any failures or bias, and adapt models as requirements and environments evolve (which we covered in Chapters [6](#) and [9](#)).
6. *Business analysis*: Developers evaluate model outcomes against business goals, to ensure that the project generates business value. This stage is closely related to project scoping.

[Figure 10-1](#) depicts these stages.

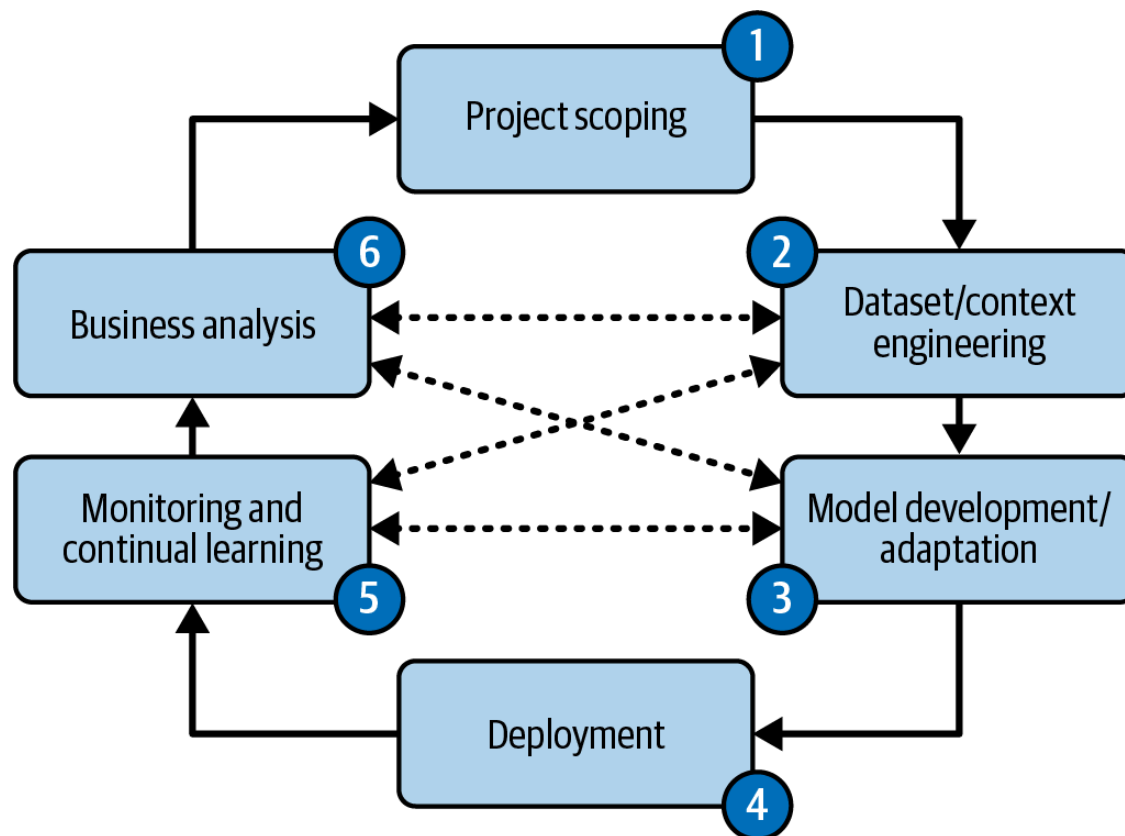


Figure 10-1. The iterative process for developing AI systems, adapted from *Designing Machine Learning Systems* by Chip Huyen

Model development is at the core of ML engineering. In AI engineering, however, you are working with a model that is already trained and adapting it to your own needs. Prompt-based techniques can adjust model output without updating model weights. Some adaptations aren't possible without changing model weights; for example, you can't adapt a model to a new task it wasn't exposed to during training. In such cases, you should look into model fine-tuning techniques.

The process of *dataset engineering* is relatively straightforward in traditional ML. In the most common scenario, it involves working with tabular data and predefined model outputs. For example, for hotel booking cancellation, the outputs would be *cancelled* or *not cancelled* within a month of the reservation date. In the case of AI agents, outputs are not restricted to a fixed set of labels, and annotating them often requires you to exercise subjective judgment or have domain expertise.

We've noticed that dataset (context) engineering in AI engineering would therefore potentially involve these additional steps, depending on the approach being used:

Prompt engineering

In an ideal scenario, this requires a fixed benchmark dataset of inputs and expected outputs to measure and optimize prompt performance.

Retrieval-augmented generation (RAG)

This focuses on preparing, chunking, and embedding domain-specific corpora.

Fine-tuning

This applies in rare cases, and it emphasizes data collection, cleaning (removing duplicates and toxic or sensitive data), and annotation.

When it comes to *model deployment*, traditional ML deployment typically involves packaging a model into a REST API or embedding it in a business system. AI deployment introduces unique challenges, such as managing context length and retrieval efficiency, integrating guardrails, and orchestrating multiple models for fallback. Inference optimization is a concern in both ML and AI, but it is far more pressing for AI applications, where costs and latency can escalate quickly.

Monitoring in ML typically means tracking relevant model and business metrics. For AI systems, monitoring is harder because outputs are open ended and subjective, which means *human evaluation* often becomes necessary.

Business analysis in AI systems is broader and less clearly defined than in traditional ML. Instead of fixed metrics, teams must align on *what “good” looks like*, often across subjective dimensions like helpfulness, correctness, and safety. Evaluation typically combines automated metrics, human judgment, and user behavior signals. Because requirements often evolve through real user interactions, business analysis becomes a *continuous, iterative process* that’s closely tied to both evaluation and system improvement.

These stages are tightly interconnected and form a continuous, iterative lifecycle. While deployment and monitoring (stages 4 and 5) are the primary focus of MLOps, their success depends heavily on disciplined practices established in the earlier stages.

Next, we will identify which lifecycle stages each of the high-risk requirements (Articles 9–15) impacts and how the Databricks platform can automate compliance.

NOTE

The author is not a lawyer, and this content does not constitute legal advice. It is intended for technical and operational guidance only.

Article 9: Risk Management System

Article 9 of the EU AI Act requires providers of high-risk AI systems to establish, implement, document, and maintain a risk management system throughout the entire lifecycle of the system. The focus is on risks that can be reasonably mitigated or eliminated through the design, development, and operation of the system.

Risk management is not a one-and-done compliance check; it's a continuous and iterative process that mirrors the iterative nature of the AI lifecycle.

Project scoping (risk identification)

During *project scoping*, teams should do the following:

- Identify and analyze known and reasonably foreseeable risks to people's health, safety, and fundamental rights.
- Consider whether the AI system may adversely impact vulnerable groups (e.g., minors) under the intended use.
- Evaluate risks not only under intended use but also under reasonably foreseeable misuse.

Development and testing (mitigation)

Before a team can put a high-risk system into service, it must perform testing to determine which risk mitigation measures are appropriate and verify that the system performs consistently for its intended purpose. To comply with the regulations, they must do the following:

- Test against the predefined evaluation metrics and probabilistic thresholds to ensure testing is objective and traceable.
- Perform real-world testing where appropriate.

Post-market monitoring (via a feedback loop)

Teams must continuously analyze data collected from real-world usage to detect new or unforeseen risks that were not identified during development. These insights must feed back into the risk management process to update mitigation measures as the system evolves.

Types of risks and mitigation strategies

Now, let's walk through some examples of risks in AI systems and how to mitigate and eliminate them. They include the following:

- Data-related and model behavior risks
- Operational risks
- Security/cyber risks

Data-related and model behavior risks

Data risks are the most common source of AI failure, and they often lead to biased or unsafe model outcomes. Common risks include the following:

- Biased datasets (for example, medical data that predominantly represents male patients)
- Incorrect or inconsistent labels
- Missing, duplicated, or outdated data
- *Data drift*, in which the data no longer represents the real world

Data-related risks often become *model behavior risks*. Models trained on problematic datasets are likely to produce unsafe or discriminatory outcomes. To mitigate these risks, teams must continuously validate data quality.

Lakeflow SDPs allow teams to define data quality rules (“expectations”) that automatically detect and quarantine invalid or unexpected records. In addition, teams can set up Databricks Data Quality Monitoring on input data, to compute profile and drift metrics over time.

Teams can also extend monitoring to inference data (logged predictions and features), as we discussed in [Chapter 6](#), to enable detection of drift between training and serving. They can also configure alerts by defining SQL queries on the metrics tables, to notify teams when data deviates from expected patterns. This enables early detection of issues, before they propagate into model behavior.

Operational risks

These arise not from data or model behavior but from the processes used to move models into production. Examples of situations that create operational risks include the following:

- Broken or untested code being pushed directly to production
- Development and production environments being misaligned
- A deployment failing midway, leaving a system in an inconsistent state
- Teams accidentally deploying the wrong model version
- Teams making manual changes in production that bypass governance and auditability

These situations can lead to outages, incorrect predictions, and loss of traceability, all of which are unacceptable for high-risk AI systems.

To mitigate operational risk, deployment must follow controlled, automated, and auditable workflows. Code and configuration should be promoted to production only through CI/CD pipelines, not via manual changes in the production environment. Developers should not have write access to production, and promotion should require approval gates.

We discussed best practices in detail in [Chapter 5](#), and we also demonstrated how DABs help to simplify and streamline the deployment process to different environments.

Security and cybersecurity risks

Security risks arise when an AI system or its infrastructure can be accessed, manipulated, or misused by unauthorized parties. These risks apply not only to the model but also to data, serving endpoints, credentials, CI/CD pipelines, and even third-party dependencies. Examples of security risks include the following:

- Prompt injection or jailbreaking attempts in LLM applications
- Leakage of personal or sensitive data through logs or model outputs
- Exposure or theft of secrets (API keys and OAuth tokens) used within AI systems
- Compromise of a package dependency or CI/CD pipeline, which can lead to malicious code execution

To mitigate these risks, least-privilege access and controlled permission boundaries are essential. In [Chapters 4](#) and [9](#), when discussing *OAuth authentication*, we highlighted the importance of using the principle of minimum permissions: each service principal should have only the exact access required; for example, they should be able to query a single Model Serving endpoint and nothing else.

In [Chapters 4](#) and [9](#), we talked about the Unity AI Gateway, which helps reduce security and misuse risks by enabling guardrails (input/output filtering), rate limits, usage tracking, and inference logging. Usage is written into system tables, so teams can detect unusual traffic patterns or potential abuse. *Inference tables* provide detailed logs of every request, and *MLflow Tracing* captures all intermediate steps for LLM applications, enabling teams to audit and investigate suspicious activity.

The recommended best practice is to store credentials in *Databricks secret scopes* and reference them at runtime, to ensure that no secrets are embedded in code or configuration files. We used Databricks secret scopes in [Chapter 9](#).

Security risks are not limited to application behavior. They can originate from the software supply chain itself. Attackers increasingly use or exploit the following:

Typosquatting

This involves creating malicious packages with names that are very similar to popular libraries (e.g., installing “reqeusts” instead of “requests”).

Malicious or compromised packages on PyPI

A real-world example of such a package is the [fastapi-toolkit package on PyPI](#). It was compromised after attackers gained access to the maintainer’s account and published a new version containing malware that was designed to exfiltrate environment variables and credentials from build pipelines.

Malicious transitive dependencies

For example, the LiteLLM library (which has about 100 million downloads per month)—a dependency for CrewAI, databricks-agents, and other Python packages—recently came [under attack](#).

Repojacking/dependency hijacking

In GitHub Actions, if an Action is referenced as `{owner}/{repo}@{tag}` or `{branch}` and attackers take control of the repository, they can inject malicious code. Then, when the workflow runs, the runner loads secrets into memory that [malware can exfiltrate](#).

Users accidentally committing API keys or OAuth tokens to Git

This is one of the most common and avoidable security incidents.

These are only a small number of examples. As AI adoption accelerates and introduces new attack surfaces in the coming years, we will inevitably see more frequent and sophisticated attacks. To mitigate these risks, companies can consider using *private package repositories* and allowing installation only from approved sources instead of the public PyPI.

They can also avoid referencing GitHub Actions by branch or tag and instead pin them to a *commit SHA* to prevent repojacking.

These examples demonstrate how a small misconfiguration or dependency oversight (core or transitive) can lead to severe consequences. Code scanning across organizations can help to find and address vulnerabilities, to ensure best practices are followed.

Article 10: Data and Data Governance

While Article 9 of the EU AI Act requires a continuous risk management process, Article 10 focuses on one of the most critical risk areas: the *quality of data* used to develop high-risk AI systems. If an AI system involves training a model, the provider must ensure that training, validation, and testing datasets are governed, managed, and controlled throughout the lifecycle.

Let's walk through the AI project lifecycle and requirements that teams should consider at each stage.

Project scoping

Compliance starts before any data is ingested. During project scoping, teams must understand the intended purpose of the AI system and establish a *data governance strategy* aligned with Article 10. This includes defining how data will be collected, what sources will be used, and whether the data is suitable for the model.

Article 10 requires that organizations do all of the following:

- Document the design choices that led to selecting specific datasets
- Record assumptions about what data represents
- Assess whether data is sufficiently available, representative, and suitable
- Consider the potential impact of data biases on individuals or groups

At this stage, governance is primarily about *planning and accountability*: identifying data sources, evaluating licensing and usage rights, and as-

sessing whether the system may affect vulnerable populations.

Dataset/context engineering

In this phase, raw data is processed into structured and governed datasets. Article 10 mandates data governance practices and explicitly requires documentation of the following:

- Data collection processes and origin
- Data preparation operations, such as labeling, cleaning, and aggregation
- Analysis of potential biases and measures to mitigate them
- Identification of gaps in representativeness or quality and how they will be addressed

Datasets must be all of the following:

- Relevant
- Sufficiently representative (including geographic or contextual characteristics of the real-world deployment setting)
- To the greatest extent possible, free of errors
- Complete for the intended purpose

Databricks capabilities help streamline data quality enforcement. Built-in [expectations](#) allow you to automatically detect and isolate invalid records.

However, this functionality is limited to declarative pipelines. For standard PySpark-based workflows, an alternative is [DQX](#) by Databricks Labs. It is a data quality framework for Apache Spark that enables teams to define, monitor, and handle data quality issues directly in Python-based pipelines. While we do not explore DQX in detail in this book, it is a valuable project that teams working with PySpark-based data pipelines will find worth exploring.

Delta Lake provides data version control and lineage tracking: every transformation step becomes traceable and auditable. *Unity Catalog* sup-

ports best governance practices by providing the least-privilege access that's required for the purpose. We discussed best practices in [Chapter 5](#).

Compliance is demonstrated by traceability (how the dataset was constructed) and justification (why it is suitable).

Model development/adaptation

Article 10 requires datasets that are used for model training, validating, and testing to meet the established quality criteria: governance, representativeness, and bias management.

During this phase, teams must do the following:

- Track the dataset versions they use to train each model iteration
- Validate that training, validation, and test sets represent the real-world population
- Test the model on relevant subgroups to verify fairness and prevent discriminatory outcomes

In [Chapter 3](#), we demonstrated how to log evaluation results and datasets using MLflow and link it to the models registered in Unity Catalog.

Deployment

The EU AI Act requires that data within the deployment environment must remain consistent with the quality and representativeness standards we established earlier: datasets must remain relevant and appropriate to the intended purpose.

Practical safeguards include the following:

- Quality gates in ML pipelines that prevent model promotion if dataset or fairness metrics regress
- Controlled release processes that prevent the accidental use of outdated or unapproved datasets

Lakeflow Jobs allows for if-else statements, as shown earlier in [Chapter 5](#), to enable model promotion only if certain quality thresholds are met.

Monitoring and continual learning

Article 10 does not end at deployment. It requires organizations to continuously evaluate whether the data flowing into their systems remains representative over time. Real-world datasets change due to market shifts, user behavior changes, and seasonal effects. A model trained on compliant data may become noncompliant if the data context shifts.

Databricks supports this phase through the following:

- *Data Quality Monitoring*, which detects drift between inference data and a baseline, and between different windows of inference data
- Automated *alerting* when statistical properties deviate from expectations
- *Inference tables* and *MLflow Tracing*, which capture model inputs and outputs for audit and investigation
- Bias monitoring in Data Quality Monitoring (classification)

This fulfills the implicit requirement of Article 10: compliance is *ongoing*, not a one-time validation exercise.

Business analysis

There are rare circumstances in which processing of special-category personal data (e.g., race, ethnicity, health data) is required to detect or correct bias. Article 10.5 permits this only when all of the following are true:

- No alternative data can achieve the same objective.
- The data is strictly access-controlled and documented.
- The data is deleted once bias mitigation is completed.

Business analysis is responsible for justifying this exceptional processing and documenting why it is necessary. *Unity Catalog* permission controls, data retention policies, audit logging, and column masking support the strict safeguards mandated by Article 10.5.

Article 11: Technical Documentation

Article 11 requires providers of high-risk AI systems to prepare and maintain *technical documentation* that demonstrates compliance with the law. This documentation must be complete before a provider can place a system on the market or put it into service, and it must be kept continuously up to date as the system evolves.

The purposes of technical documentation are to prove that an AI system meets all applicable regulatory requirements (such as risk management, data governance, and transparency) and to enable authorities to perform independent assessments of the AI system.

Article 11 specifies that technical documentation must include all elements defined in the Annex IV reporting framework, at minimum. Annex IV requires the documentation of information such as the following:

- Intended purpose and system architecture
- Dataset characteristics and data governance measures
- Risk management strategy and mitigation actions
- Model performance evaluation, limitations, and monitoring processes
- System logs, versioning, and auditability records

Importantly, this is not technical documentation in the traditional engineering sense. It is a *compliance artifact* that describes how the system meets the Act's requirements and provides evidence to support that claim.

Technical documentation becomes the compliance backbone—the place where all actions taken under Article 9 (Risk Management) and Article 10 (Data Governance and Data Quality) are captured, evidenced, and made auditable.

With the growing adoption of AI-assisted software development, documentation is no longer just a compliance requirement; it is becoming a *technical necessity*. Tools that generate or modify code based on prompts, architecture descriptions, or past decisions require context to operate correctly. Without documentation, there is no shared understanding of

why decisions were made, what constraints exist, or what assumptions must be preserved.

Documentation should *live where the system lives: in the repository, under version control*, not in scattered Confluence pages or slide decks. One effective approach could be to store documentation as Markdown files in the same repository as the code. It could be updated iteratively, ideally in the same pull requests as code changes, and AI-assisted tools like Claude Code could help generate or refine these updates.

Keeping documentation in the repo provides several benefits like *traceability* (every change to code and documentation is tracked together), *discoverability* (other developers can easily find relevant documentation), and *governance* (documentation becomes part of code review, not an afterthought).

Article 12: Record-Keeping

Article 12 requires high-risk AI systems to automatically record events in logs for their entire operational lifetime. Logging is not an optional enhancement; it is a mandatory capability that must be built into the system from the beginning. The system must provide an audit trail that makes it possible to reconstruct what happened, why it happened, and who interacted with it.

Logs must capture the events that are necessary to the following:

Detect situations that may present risks

Logs must identify events that could endanger health, safety, or fundamental rights. For example, they must identify anomalous predictions, data drift, and system malfunctions.

Detect substantial modifications

Logs must record evidence of changes that alter the system's behavior or compliance status, such as retraining on new data, modifying features, and deploying a new model version without approved validation.

Support post-market monitoring (providers only)

Providers must continuously monitor deployed systems to confirm ongoing compliance, detect emerging risks, and demonstrate in their logs that safeguards remain effective.

Monitor operational behavior and oversight (deployers only)

Deployers must ensure and document in their logs that their systems operate safely day-to-day, that human-in-the-loop processes are working, and that every decision can be traced to its origin.

Rather than traditional application logging, this regulation focuses on logs that help explain *decision-making behavior*, not just technical errors. The logging must be “appropriate to the intended purpose,” meaning logs should make it possible to trace a specific output back to its inputs, the model version used, and any relevant human intervention.

For high-risk systems used in *remote biometric identification or categorization*, where permitted by law, additional requirements apply. These systems must record the following:

- When the system was used (the start and end time of each session)
- Which reference database was used for comparison
- Which inputs resulted in a match
- The identity of the human reviewers who verified or approved results

To meet Article 12 obligations, AI systems should automatically log the categories of events listed in [Table 10-2](#)).

Table 10-2. Required logs and traceability per event category

Event categories	Examples of captured data
Model lifecycle	Model name, registered model version, training run ID, and when the model was retrained
Inference activity	Timestamp, request ID, input features, system output, (when applicable) confidence levels, latency, and error codes
Data sources	The exact version of the dataset used for training and validation, schema, and data drift metrics
Code/deployment source	Git commit hash and deployment pipeline run ID
System and infrastructure	Cluster/job ID, serving endpoint, environment variables, and package versions
Human oversight	Reviewer ID, review timestamp, action (take decision/override), and reason codes
Security and access	User ID, workspace access logs, API calls, and configuration changes

The following Databricks features provide a solid foundation for implementing Article 12 requirements:

Inference tables

These automatically record every model invocation (inputs, outputs, model version, latency, and status) to create a structured audit trail for each prediction.

System tables

These capture activities such as job runs, endpoint invocations, and data access for operational oversight.

Data Quality Monitoring

This continuously computes data drift, bias, and performance metrics to detect new or evolving risks, and it supports alerts for post-

market monitoring.

MLflow Tracking and Model Registry

These store training metadata, datasets, parameters, metrics, and model versions to detect substantial modifications and maintain model lineage.

MLflow Tracing

This records intermediate steps and reasoning chains in complex or multicomponent AI systems.

In Chapters [3](#) through [5](#), we established the foundations of traceability and reproducibility in ML projects. We demonstrated how to connect every component—code, data, infrastructure configurations, and model artifacts—to ensure that every deployment is fully auditable. Then, in [Chapter 6](#), we focused on the monitoring of ML systems. Finally, in Chapters [7](#) through [9](#), we expanded these principles to the unique challenge of AI systems, focusing on how to maintain traceability, reproducibility, and observability within GenAI workflows.

Article 13: Transparency and Provision of Information to Deployers

Article 13 extends the principles of Article 11. While Article 11 focuses on *technical documentation* for regulators, Article 13 ensures *transparency for deployers*—the organizations that actually operate high-risk AI systems. The goal is simple: deployers must be able to *understand, interpret, and appropriately use* the system’s outputs.

Transparency must be built into a system from the start. This means more than exposing algorithms; it ensures that users can recognize the system’s capabilities and limitations and have confidence in its results. Without this, human oversight (Article 14) and risk management (Article 9) cannot function effectively.

Every high-risk AI system must be accompanied by *clear, accessible instructions for use* in digital form. These must describe the following:

- The provider's identity and contact details
- The system's intended purpose, accuracy, robustness, and cybersecurity characteristics
- Known limitations and foreseeable risks
- How outputs should be interpreted and under what conditions they may be unreliable
- Data or input requirements
- Human oversight expectations
- Maintenance needs and software update procedures
- How to collect and interpret logs

The *provider* is responsible for supplying this information. The *deployer* must follow it, to ensure that the system is used as intended. If a *deployer* alters the system's design, retrains the model, or changes its purpose, it effectively becomes a *provider* and assumes full compliance obligations, including risk assessment and conformity evaluation.

Transparency thus links design and operation: providers make systems explainable, and deployers keep them understandable in practice. It ensures that even complex AI remains interpretable, accountable, and safely governed throughout its lifecycle.

Article 14: Human Oversight

Article 14 introduces one of the most practical and human-centered requirements of the EU AI Act: every high-risk AI system must be designed so that it can be effectively overseen by natural persons during its operation.

Human oversight is not an optional safeguard. It is a regulatory requirement that ensures AI systems remain controllable, accountable, and aligned with human judgment. The goal is to prevent or minimize risks to health, safety, and fundamental rights. It serves as the final line of defense when automated systems make decisions that could impact individuals or society.

Human oversight under the AI Act requires systems to be designed with *appropriate human-machine interfaces* that allow trained personnel to do the following:

- Understand the system’s capabilities and limitations
- Detect anomalies or unexpected behavior
- Remain aware of *automation bias*
- Correctly interpret outputs in context
- Override or disregard AI-generated decisions when necessary
- Interrupt or safely stop a system’s operation if it behaves unexpectedly

These oversight measures must be *proportionate* to the system’s risk level, autonomy, and intended use. For example, in low-frequency, high-impact decisions (such as medical diagnoses or credit approvals), human review may be mandatory before each decision is finalized. In higher-volume operational systems (such as predictive maintenance), periodic audits or threshold-based alerts may be sufficient.

For biometric identification systems covered under Annex III(1)(a), Article 14 adds a further safeguard: no identification result may be acted upon unless verified by at least two qualified human reviewers (except where law enforcement exemptions apply).

Human oversight can be implemented by both *providers* and *deployers*, depending on the system design. Providers should enable technical controls and interfaces embedded into the system (for example, dashboards, alerts, or review workflows) to make human monitoring and intervention possible. Deployers should enable organizational or procedural controls (such as human review steps, approval protocols, or secondary validation processes) before executing actions.

When deploying AI applications on Databricks, organizations can embed several human-in-the-loop safeguards into the operational workflow. We discussed one of them in [Chapter 5](#). At the code level, organizations can enforce branch protection rules in GitHub so that all changes to the main branch require approval by at least two reviewers before merging. Organizations can also enforce deployment protection rules so that deployment to production can only proceed after human approval.

MLflow provides an interface for manual review of model outputs. It enables reviewers to inspect predictions, approve or override them, and log decisions for audit and traceability.

Article 15: Accuracy, Robustness, and Cybersecurity

Article 15 brings together three essential pillars of trustworthy AI: *accuracy, robustness, and cybersecurity*. It also requires high-risk AI systems to maintain these qualities *consistently throughout their lifecycle*. It defines what the system itself must *technically achieve* to be considered safe and compliant.

A high-risk AI system must not only perform well under test conditions but also remain reliable, stable, and secure once deployed, even as its data, environment, and users evolve over time.

Accuracy represents the degree to which the system's predictions or classifications correspond to reality or the intended output. Under Article 15, providers must do the following:

- Define and document the accuracy metrics that are appropriate to the system's purpose
- Validate performance against those metrics during development and testing
- Disclose those metrics in the *instructions for use*

The “appropriate” level of accuracy depends on the system's context and risk. For instance, an AI-based medical triage tool or credit-scoring model must achieve far higher validated accuracy and lower false negative rates than a document classifier or maintenance predictor.

Robustness is a system's ability to *operate reliably under varying conditions*, including imperfect data, software faults, environmental noise, and human interaction. A robust AI system must continue to function safely even in the presence of errors or inconsistencies, and it must recover gracefully if something goes wrong.

Key robustness measures include the following:

Error and anomaly handling

This includes detecting and managing unexpected inputs or missing values without failure.

Fail-safe mechanisms

These provide fallback modes or manual override when conditions deviate from expected norms.

Technical redundancy

The system must include backup models, alternate logic paths, or rule-based guards that maintain safe operation.

Continual learning safeguards

For high-risk AI systems that continue to learn after deployment, mechanisms must be in place to prevent situations where biased or incorrect outputs are reintroduced as new inputs and gradually distort the system's behavior over time.

Cybersecurity in high-risk AI systems means that such systems must be *resilient to manipulation and malicious interference*, including attacks targeting data, models, or infrastructure. This includes AI-specific vulnerabilities such as the following:

Data poisoning

This involves injecting corrupt or biased samples into training datasets.

Model poisoning

This involves compromising pre-trained components or shared embeddings.

Adversarial examples

This involves intentionally crafting inputs that are designed to make the model misclassify data.

Model evasion

This involves probing decision boundaries to reverse-engineer sensitive logic.

Confidentiality or extraction attacks

These attempt to gain access to training data or proprietary parameters.

As we noted in the discussion of Article 9, providers must implement technical and organizational measures to prevent, detect, and mitigate these threats, and to ensure that any compromise does not lead to unsafe outputs or privacy violations.

MLOps Checklist

Before teams deploy an AI system to production, especially in regulated environments, they must assess whether both the system and the surrounding processes meet required standards for safety, governance, and compliance. However, they should not treat this type of checklist as a one-time, final gate.

Instead, an MLOps checklist is most effective when applied at *key lifecycle milestones*, such as before production deployment, during major releases, or as part of periodic compliance reviews. Running it earlier helps identify gaps proactively, while revisiting it over time ensures that the system remains compliant as it evolves.

Importantly, not all criteria need to be fully satisfied from day one. Teams should treat this checklist as a *maturity framework*, prioritizing critical risks first and progressively strengthening controls as the system scales and its impact increases.

With that in mind, we can synthesize these requirements into a single actionable view. The checklist in [Table 10-3](#) illustrates how modern MLOps practices directly support compliance with the EU AI Act.

Table 10-3. MLOps checklist

Item #	Assessment criterium	Linked article(s)
Documentation		
1	The business goals, intended purpose, and KPIs of the AI system are documented and kept up to date.	9, 11, and 13
2	The roles, responsibilities, and accountability of all team members are documented and regularly reviewed.	9 and 11
3	AI system risk assessment and mitigation plans are created, maintained, and updated throughout the lifecycle.	9
4	Documentation includes assumptions, design choices, and justification for dataset selection and model type.	10 and 11
5	Technical documentation demonstrates compliance (in architecture, datasets, risk management, and performance).	11
6	Documentation resides in version control and is updated with code changes for full traceability.	11 and 12
7	Instructions for deployers describe intended use, limitations, and maintenance procedures.	13
8	Human oversight procedures and escalation paths are described in documentation.	14
9	Every release includes a versioned compliance summary or “model card.”	11 and 13
Infrastructure		
10	Infrastructure is defined as code (IaC) to ensure traceability and reproducibility.	9 and 15

Item #	Assessment criterium	Linked article(s)
11	IaC is stored in version control and updated through pull requests.	9 and 12
12	Changes to infrastructure are automatically deployed via controlled CD pipelines.	9 and 15
13	Only automated deployment pipelines can apply changes; manual deployment is restricted.	9 and 15
14	At least two environments (pre-production and production) exist for controlled deployment and are synchronized in configuration.	9 and 15
15	Infrastructure health and cost are continuously monitored; alerts are configured for anomalies.	9 and 15
Data and models		
16	Code for data gathering, cleaning, and feature preparation is version-controlled.	10 and 12
17	Datasets used for training, validation, and testing are versioned and auditable.	10 and 12
18	Dataset lineage and transformations are traceable.	10 and 12
19	Data quality rules and expectations are defined and enforced automatically.	10 and 15
20	Dataset representativeness, completeness, and bias checks are documented and validated.	10
21	Processing of sensitive or special-category data is justified, access-controlled, and deleted after use.	10 and 15
22	Drift monitoring is implemented for both training and inference data.	10 and 12

Item #	Assessment criterium	Linked article(s)
23	Offline evaluation metrics (e.g., accuracy, F1) are logged and monitored over time.	9 and 15
24	Feedback loops capture real-world performance and update the risk management process.	9 and 10
Code and deployment governance		
25	All code related to the AI system (data, model, and API) is version controlled (with a preference for a single version control repository).	9 and 12
26	Pull requests with peer review are required for all code and configuration changes.	9 and 14
27	Continuous integration (CI) runs automated tests and validation at PR creation.	9 and 15
28	Continuous delivery (CD) promotes artifacts only after passing quality and compliance gates.	9 and 15
29	Deployment pipelines facilitate recording run IDs, approvers, and linking deployed model versions to code.	12 and 14
30	Rollback procedures allow reversion to previous safe model and infrastructure states.	9 and 15
31	Developers do not have direct write access to production environments. Such access can be provided temporarily when production failures occur.	9 and 15
Logging and auditability		

Item #	Assessment criterium	Linked article(s)
32	The system automatically logs all model lifecycle events (training, retraining, and deployment).	12
33	Each inference request logs the timestamp, model version, input reference, output, and latency.	12
34	Logs capture data versions, schema, drift metrics, and alert history.	12
35	All logs are retained securely, tamper resistant, and available for audit.	12
36	Human interventions (reviews, overrides, and approvals) are logged with user IDs and reasons.	12 and 14
37	Logging covers cybersecurity events (unauthorized access, failed authentications, and data exfiltration attempts).	12 and 15
Transparency and explainability		
38	The system provides meaningful explanations for outputs (e.g., feature importance, and SHapley Additive exPlanations [SHAP]).	13 and 14
39	Model behavior and limitations are documented in language that's understandable by deployers and users.	13
40	Uncertainty estimates or confidence levels are logged and exposed when relevant.	13 and 15
41	Providers supply digital instructions describing interpretation, accuracy, and robustness.	13
42	Explanations are validated with intended users to ensure understandability.	13 and 14

Item #	Assessment criterium	Linked article(s)
43	Human approval gates are embedded in model deployment workflows.	14
44	Dashboards or review interfaces enable oversight of predictions and system health.	14
45	Oversight personnel are trained to detect anomalies and automation bias.	14
46	Humans can override or stop the system safely if behavior is unexpected.	14 and 15
47	Records of oversight actions are stored for audit and continuous improvement.	12 and 14
Accuracy, robustness, and cybersecurity		
48	Accuracy thresholds are defined and validated under representative conditions.	15
49	System performance is periodically re-evaluated with real-world data.	9 and 15
50	Fail-safe mechanisms and fallback modes are implemented for degraded states.	15
51	Regular stress, load, and penetration tests verify robustness and latency targets.	15
52	Least-privilege access and network isolation are enforced for models and data.	9 and 15
53	Secrets (API keys and tokens) are stored securely and never embedded in code.	15
54	Dependencies and supply chain components are verified and scanned for vulnerabilities.	15
55	Controls are in place to prevent adversarial, poisoning, or model-evasion attacks.	15

Item #	Assessment criterium	Linked article(s)
56	Cybersecurity controls are periodically reviewed and updated.	15
Monitoring and continual compliance		
57	Post-market monitoring collects data on system behavior and emerging risks.	9 and 12
58	Alerts are generated when performance, drift, or bias metrics exceed thresholds.	9, 10, and 15
59	Results from monitoring feed back into risk management and retraining processes.	9 and 10
60	Compliance status is periodically reviewed and reported to governance stakeholders.	9, 11, and 12

Conclusion

One of the biggest challenges organizations face with AI governance is not a lack of frameworks, policies, or principles. It's making them *practical and actionable*. Governance often lives in slide decks and policy documents, while real decisions are made in code, pipelines, and product iterations.

This is exactly where the practices covered throughout this book come together.

Every chapter has not been just about building better AI systems. It has been about building *governable systems by design*:

- When you version code, data, and models, you enable *traceability*.
- When you evaluate outputs systematically, you enable *accountability*.
- When you monitor systems in production, you enable *risk detection*.
- When you design feedback loops, you enable *continuous oversight*.

- When you standardize pipelines, you enable *repeatability and auditability*.

As you move forward, consider governance as part of your system design—not an afterthought. Build processes that scale, document decisions that matter, and foster a culture where ethical considerations are treated as first-class engineering requirements.

In the end, AI governance is about trust. And trust, once earned through consistent and responsible practice, becomes a lasting competitive advantage.

1 European Law Institute, “The Concept of ‘AI System’ under the New AI Act: Arguing for a Three-Factor Approach,” Commission Guidelines on the Application of the Definition of an AI System and the Prohibited AI Practices Established in the AI Act, 2024, <https://oreil.ly/BGtNv>.

2 Future of Life Institute, *High-Level Summary of the AI Act*, February 27, 2024, <https://oreil.ly/uTGLH>.

Previous chapter

< [9. Deploying in Monitoring LLM-Based Systems](#)

Next chapter

[Epilogue](#) >